Automation Project Proposal

Training Buddy — a Claude Code Plugin That Turns Any Training Plan into a Guided, Assessed Learning Loop

Ad-hoc Activity Plan, Item 4 · Prepared by Leander · 2026-08-24

Problem

Training and growth plans are run by hand, per person: the EM writes the plan, the mentee self-studies, and assessment and progress tracking are manual. Meanwhile the naive way to use AI for learning — "Claude, teach me X" — fails in a specific, predictable way: **it hands over answers.** That optimizes for finishing the task, not for being able to reproduce the reasoning later without the AI in the room — which is exactly what a graded assessment, or a client conversation, tests.

Proposed project

During my 2-week training plan I solved this with a written working agreement and five custom Claude Code commands. They ran the whole plan — but they are hardcoded to me and to my topic. The project is to generalize and package that system as **Training Buddy**: an installable Claude Code plugin any Full Scale engineer can point at their own training plan.

Command	What it automates
`/day-start`	Orientation from the plan file — today's objective, prerequisites, time budget — plus a warm-up quiz on yesterday's material.
`/quiz`	Socratic quiz, one question at a time, **answers withheld**. Escalates recall → application → "what breaks if…". Every miss is logged.
`/explain-back`	Feynman drill: the learner explains a concept in their own words; Claude grades it and names the gaps.
`/design-drill`	Timed design prompt, then a critique against a rubric.
`/day-end`	Verifies the day's evidence exists on disk, updates the status file, drafts the EOD report.

Two assets ship with the commands, and they are what made the loop actually work:

- **The buddy contract as plugin rules** — no implementation code until the learner states an approach first, open design questions get turned back into questions, quiz comes *before* the build session. This is the discipline vanilla AI usage lacks.
- **A persistent weak-spot list** — every quiz miss is logged (what was answered vs. what is true), and retest prep drills that list instead of re-reading everything.

Progress stays EM-visible without any new tooling: each day produces a committed evidence trail (journal, deliverables, EOD draft) — the same pattern my plan used for its daily reporting.

Why this project

The prototype predates the proposal. This system ran my full 10-day plan end-to-end, and the evidence is in the training-plan repo: the five commands, the working agreement, a weak-spot list that tracked 21+ gaps across the plan, and a final retest answered with a question-by-question comparison against the Day 1 baseline. The project is extraction, generalization, packaging, and documentation — not invention. It also fits where the company is already heading: Claude masterclasses and hands-on workshops are running now, and this gives engineers a concrete, disciplined way to apply Claude to their own growth areas — an AI learning activity that speeds up learning, per the activity item.

Phased scope

Phase	Scope	Depends on
MVP	Extract the commands and contract, make them plan-agnostic (plan file as input, generic weak-spot and status format), package as an installable Claude Code plugin with documentation and an example plan. Pilot on one other engineer's growth area.	Nothing — all material exists.
Phase 1.5	The EM side: a plan-authoring command that generates a plan skeleton from a growth area and assessment format, plus a per-mentee weak-spot/progress rollup — trimming the hand-run part of training plans.	MVP pilot feedback.
Phase 2	Growth Portal integration: pull course and objective context so `/day-start` can point at relevant GP courses; optionally post completion or comprehension signals back to GP.	A GP API being available.

Risks and mitigations

- **Learners can bypass the contract** by asking vanilla Claude. The plugin cannot prevent that — instead it makes the disciplined path the convenient path, and the evidence trail (quiz logs, weak-spot list) shows whether the loop was actually run.
- **Overfit to my plan and topic.** The MVP explicitly includes a pilot with a different engineer on a different topic; generalization is not done until that runs clean.
- **GP integration may not be possible.** It is phased last, and the plugin is fully useful without it.

Proposed next step

If the direction looks right, I would start the MVP — roughly a one-week time-box at the same daily hours as the course work — and I would like your help picking a pilot: one engineer with an active

growth area willing to run a short plan through the plugin. I will propose a concrete schedule once the scope is agreed.
